

$$S_n = 1 + t + t^2 + t^3 + \dots + t^{n-1} = \frac{1-t^n}{1-t}$$

$$S_\infty = 1 + t + t^2 + t^3 + \dots \infty = \frac{1}{1-t} \quad (t < 1)$$

$\frac{d}{dt}$ on both sides

$$0 + 1 + 2t + 3t^2 + 4t^3 + 5t^4 + \dots \infty = \frac{d}{dt} \left(\frac{1}{1-t} \right)$$

$$= \frac{-1}{(1-t)^2} \times (-1)$$

$$= \frac{1}{(1-t)^2}$$

$$S = 1 + 2t + 3t^2 + 4t^3 + \dots \infty \quad (t < 1)$$

$$tS = t + 2t^2 + 3t^3 + 4t^4 + \dots \infty$$

$$S - tS = 1 + t + t^2 + t^3 + \dots \infty$$

$$S(1-t) = \frac{1}{1-t}$$

$$\Rightarrow \boxed{S = \frac{1}{(1-t)^2}}$$

$$\frac{d}{dx} \left(\frac{1}{x} \right) = \frac{d}{dx} x^{-1} = -1x^{-2} = -\frac{1}{x^2}$$

Amortized running time for insertLast

C_i - cost of inserting the i^{th} item

$$C_i = 1 + \begin{cases} i-1 & \text{if } i-1 = 2^k \\ 0 & \text{otherwise} \end{cases}$$

$$\text{Amortized RT} = \frac{\sum_{i=1}^n C_i}{n} = \frac{\sum_{i=1}^n 1 + \sum_{j=1}^{\lfloor \log_2(n-1) \rfloor} 2^j}{n}$$

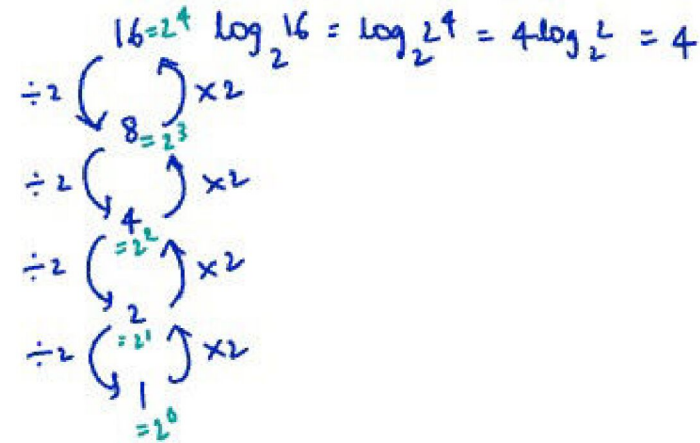
$$= \frac{n + 2 \times (2^{\lfloor \log_2(n-1) \rfloor} - 1)}{n}$$

$$= \frac{n + 2(n-1-1)}{n} = \frac{n+2n-4}{n}$$

$$= \Theta(1)$$

$$a^{\log_a n} = n$$

is n to $\log_2 n$
 2^n to n



$$C_1 + C_2 + \dots + C_{17}$$

$$\downarrow$$

$$1 + 16$$

$$+ 0 + 0$$

$$C_1 + C_2 + \dots + C_{16} + C_{17} + C_{18} + C_{19}$$

$$\# \text{ powers of } 2 = \lfloor \log_2(19-1) \rfloor$$

$$2 + 4 + 8 + 16$$

$$2^1 + 2^2 + 2^3 + 2^4$$

$$2^1 + 2^2 + 2^3 + \dots + 2^k =$$

$$\frac{a(1-r^n)}{(1-r)} = \frac{2(1-2^k)}{(1-2)} = 2(2^k - 1)$$

$$r=2$$

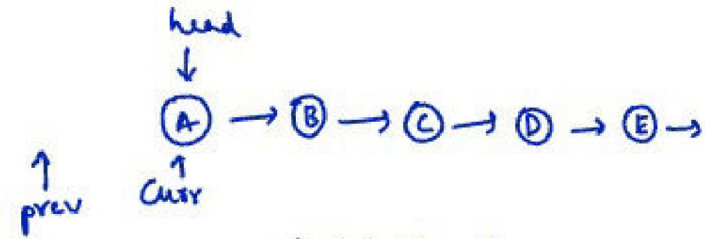
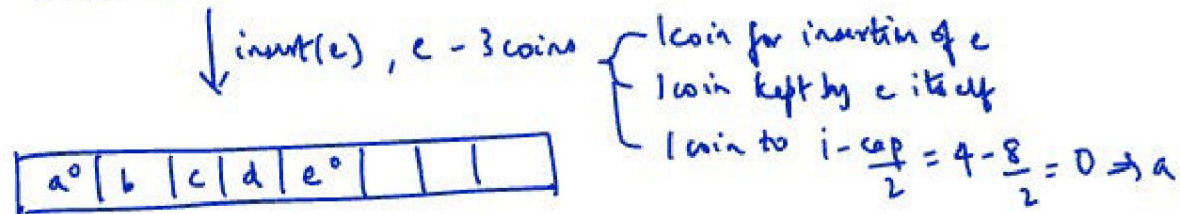
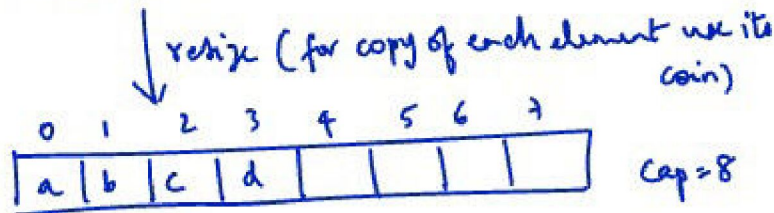
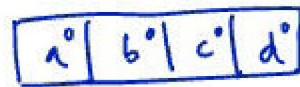
$$a=2$$

$$n=k$$

• Banker's method:

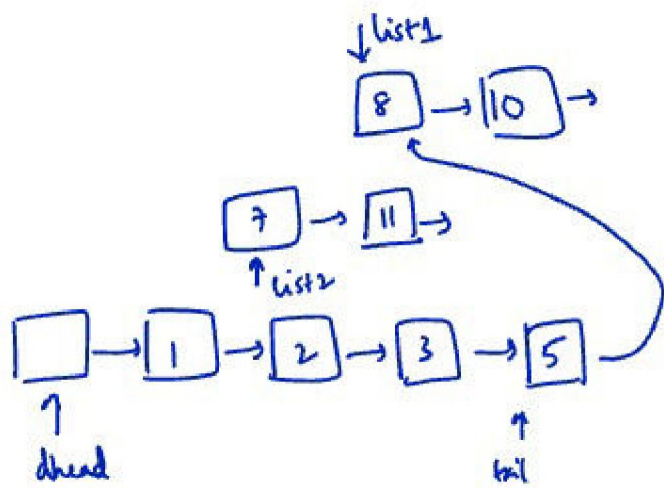
- Every element we insert gets 3 coins which could be used for doing work
- 1 coin - work for raw insertion (at index i)
- Keep 1 coin with the element
- Give 2nd coin to the element at $i - \text{cap}/2$

Base case
(each element has atleast one coin after its insertion)



start of iteration:

- we have curr & prev



["cat", "dog", "bat"]

• RT for checking if the list contains a string $O(n)$

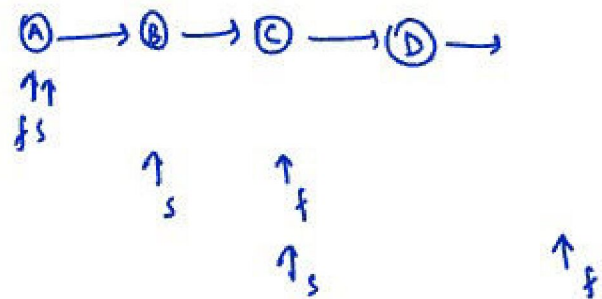
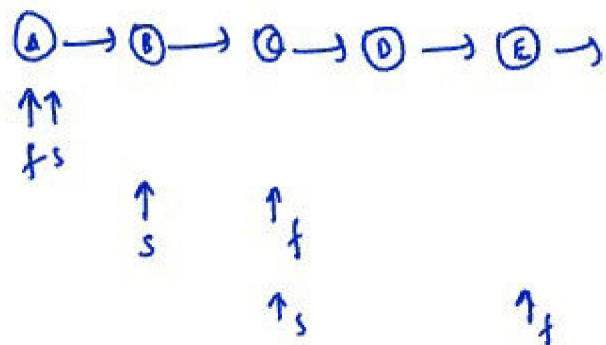
Set / HashSet / unordered-set → checking if the string exists
 (python) (Java) (C++) in the set → $O(1)$ RT on average.

keys

HashMap is the datastructure that enables average constant time lookup

Dictionary / HashMap / unordered_map → constant time for the following ops
 (python) (Java) (C++)

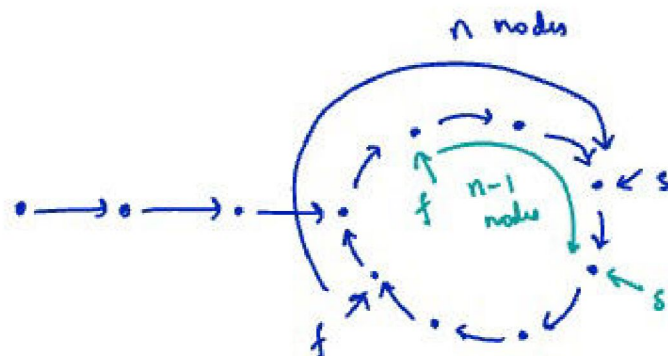
1. adding a new (k,v) pair
2. checking if 'k' already exists
3. updating value corresponding to a key.
4. retrieving value corresponding to a key



```

fast != null && fast.next != null
while (fast.next != null && fast != null) {
    slow = slow.next;
    fast = fast.next.next;
}

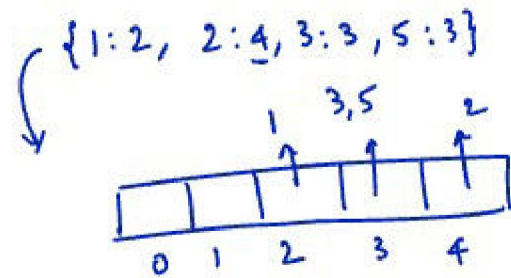
```



Observations:

1. Once a pointer (f/s) enters the cycle, it never exits.
2. Since it is a finite list, both pointers would enter the cycle.
3. The dist from f to s keeps reducing by 1 in every iteration. $\therefore$ They have to meet.

[1, 2, 3, 3, 2, 2, 1, 3, 2, 5, 5, 5]



1 → 2 → 3 → 4 → 5 →

1 → 2 → 3 → 4 → 5 →

5 → 4 → 3 →

1 5 2 4 3

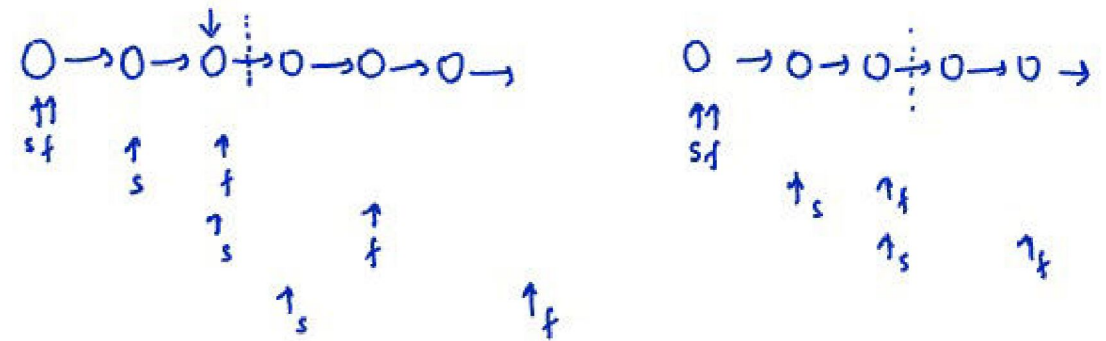
1 → 2 → 3 → 4 → 5 →

5 → 4 →

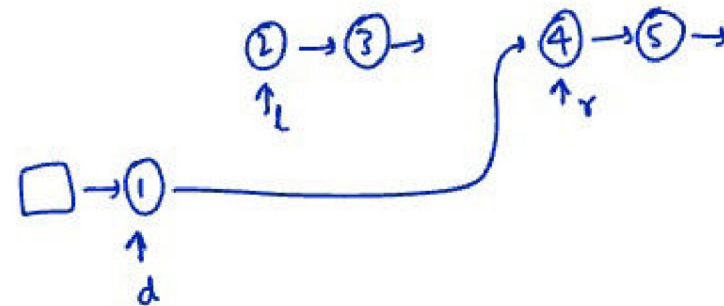
1 → 5 → 2 → 4 → 3 →

split:

- split list into 2 halves & return head of the right list.
- If odd #elements, left should have one extra
- We can assume atleast 3 nodes



while (fast.next != null && fast.next.next != null)

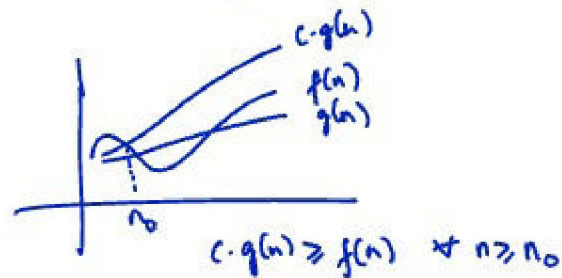


$O(n)$ - upper bound asymptotically

- constants & lower order terms don't matter

$$f(n) = O(g(n)) \text{ iff } \exists c, n_0 \text{ s.t. } f(n) \leq c \cdot g(n) \quad \forall n \geq n_0$$

definition



$f(n) = 3n^2 + 4n + 6 \quad f(n) = O(n^2)$

$$3n^2 \leq 3n^2 \quad \forall n \geq 1$$

$$4n \leq 4n^2 \quad \forall n \geq 1$$

$$6 \leq 6n^2 \quad \forall n \geq 1$$

$$3n^2 + 4n + 6 \leq 3n^2 + 4n^2 + 6n^2 = 13n^2$$

$$3n^2 + 4n + 6 \leq \frac{13}{c} n^2 \quad \forall n \geq \frac{1}{n_0} \Rightarrow \boxed{f(n) = O(n^2)}$$

$3n^2 + 4n + 6 = O(n^3) \checkmark$
 $= O(n^3 \log n) \checkmark$
 $= O(n^2 \log n) \checkmark$
 $= O(n \log n) \times$

$$\log_b n = \frac{\log_{10} n}{\log_{10} b}$$

$$1 < \log n < n < n \log n < n^2 < n^3 < \dots < n^{1000} < 1.1^n$$

$$1.1^n = O(n^{1000})$$

$\Omega(n)$ - lower bound

$$f(n) = \Omega(g(n)) \text{ iff } \exists c, n_0 \text{ s.t. } f(n) \geq c \cdot g(n) \quad \forall n \geq n_0$$

$3n^2 + 4n + 6: \Omega(n^2) \checkmark$
 $\Omega(n^2 \log n) \times$
 $\Omega(n) \checkmark$
 $\Omega(n \log n) \checkmark$
 $\Omega(n^3) \times$

$$3n^2 \geq n^2 \quad \forall n \geq 1$$

$$3n^2 + 4n + 6 \geq n^2 \quad \forall n \geq 1$$

$$\exists c=1, n_0=1$$

$$\Rightarrow f(n) = \Omega(n^2)$$

Orders of growth

1
 n
 $\log n$
 n^2
 n^3
 $n \log n$
 $\vdots$

Θ :

$$f(n) = \Theta(g(n)) \text{ iff } \exists c_1, c_2, n_0 \quad c_1 g(n) \leq f(n) \leq c_2 g(n) \quad \forall n \geq n_0$$

$$f(n) = \Theta(g(n)) \text{ iff } [f(n) = O(g(n)) \ \& \ f(n) = \Omega(g(n))]$$

	proc 1	proc 2 (doubly parallel)
$O(n)$	n_0	$2n_0$
$O(n^2)$	n_0	$\sqrt{2} n_0$
$O(n^3)$	n_0	$\sqrt[3]{2} n_0$
$O(2^n)$	n_0	$n_0 + 1$

$2^{n+1} = 2 \cdot 2^n$

- Code examples - running time
- Master Theorem & recurrences.

Eg: $\text{for}(i=0; i < n; i++) \left\{ \begin{array}{l} \text{sum} = \text{sum} + i; \end{array} \right\} \theta(n)$ as each loop body is constant time

$\text{for}(i=1; i \leq n; i = 2i) \left\{ \begin{array}{l} // \text{constant op} \end{array} \right\}$ $\rightarrow$ This loop runs $\sim \lg n$ times
 $\Rightarrow \text{RT of loop} = \theta(\log n)$

$\text{for}(i=n; i > 0; i = i/3) \left\{ \begin{array}{l} // \theta(n) \text{ operation} \end{array} \right\}$ $\rightarrow$ This loop runs $\sim \log_3 n$ times
 $\Rightarrow \text{RT} = \theta(n \log n)$

$\text{for}(i=0; i < n; i++) \left\{ \begin{array}{l} \text{for}(j=0; j < n; j++) \left\{ \begin{array}{l} // \text{constant work} \end{array} \right\} \end{array} \right\}$ $\theta(n) \text{ op}$

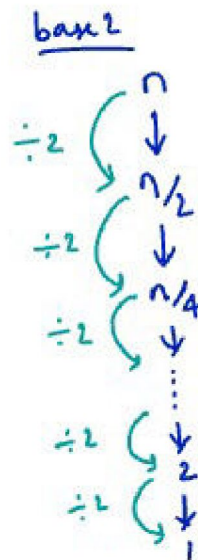
$$\text{RT} = \theta(n^2)$$

$\text{for}(i=0; i < n; i++) \left\{ \begin{array}{l} \text{for}(j=i+1; j < n; j++) \left\{ \begin{array}{l} // \text{constant op} - k \end{array} \right\} \end{array} \right\}$

inner loop work

$i=0$	$k(n-1)$
$i=1$	$k(n-2)$
$\vdots$	$\vdots$
$i=n-2$	$k(1)$
$i=n-1$	0

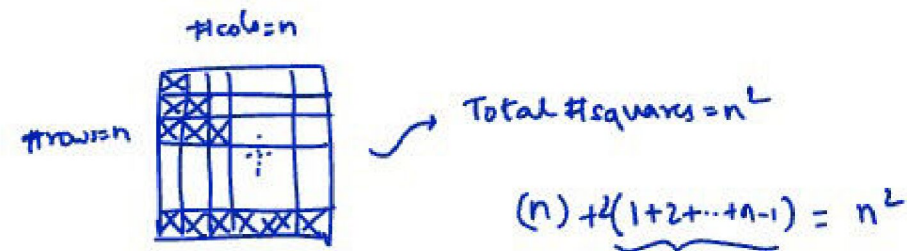
$$\begin{aligned} \text{Total work} &= k(1+2+3+\dots+(n-1)) \\ &= \frac{k(n(n-1))}{2} = \frac{k}{2}n^2 - \frac{k}{2}n = \theta(n^2) \end{aligned}$$



$\log_a a^b = b$

$16 = 2^4$
 $8 = 2^3$
 $4 = 2^2$
 $2 = 2^1$
 $1 = 2^0$

$\log_2 16 = \log_2 2^4 = 4$



$$S = 1 + 2 + 3 + \dots + (n-1) + n$$

$$\boxed{S \leq n^2}$$

$\frac{1}{2}$ of the terms $\geq \frac{n}{2} \Rightarrow S \geq \frac{n}{2} \left(\frac{n}{2} \right) \Rightarrow \boxed{S \geq \frac{1}{4}n^2}$

$\downarrow$
terms $\geq n/2$

Master theorem

```
mergesort(array): if (array.len == 0 or 1) return array;  
    left = array[:n/2]  
    right = array[n/2:]  
    { sorted-left: mergesort(left)  
      sorted-right: mergesort(right)  
    }  
    sorted = merge(sorted-left, sorted-right) // O(n) time  
    return sorted                          n - array's length
```

$T(n)$:= time taken for mergesort on
'n' sized array

$$T(1) = 1$$

$$T(n) = \underbrace{T(n/2)}_{\text{work done for mergesort on left half}} + \underbrace{T(n/2)}_{\text{work done for mergesort on right half}} + \underbrace{kn}_{\substack{\text{work done outside} \\ \text{the recursive calls} \\ \text{(merge, checking base case,} \\ \text{splitting array ...)}}}$$

$T(n) = 2T(n/2) + kn$

 Recurrence relation for merge sort

Master Theorem

$$T(n) = aT\left(\frac{n}{b}\right) + O(n^d)$$

n^d - work done outside recursion

a = # recursive calls

b = factor of subproblem size compared to the problem size

3 Cases:

Case 1: $a = b^d \Rightarrow RT = O(n^d \log n)$

Case 2: $a < b^d \Rightarrow RT = O(n^d)$

Case 3: $a > b^d \Rightarrow RT = O(n^{\log_b a})$

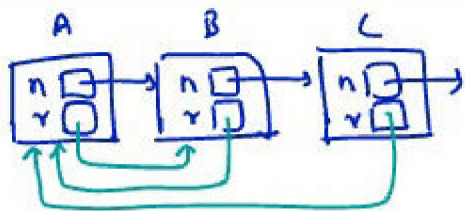
• Merge sort:

$$\begin{aligned} a &= 2 \\ b &= 2 \\ d &= 1 \end{aligned} \Rightarrow \text{Case 1} \Rightarrow RT = O(n^d \log n) = O(n \log n)$$

• Binary search

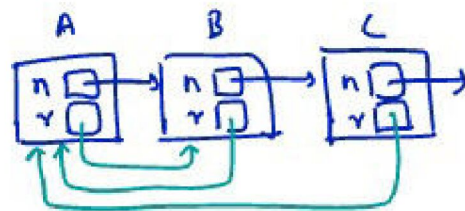
Recurrence relation: $T(n) = 1 \cdot T(n/2) + O(1)$

$$\begin{aligned} a &= 1 \\ b &= 2 \\ d &= 0 \end{aligned} \Rightarrow \text{Case 1} \Rightarrow RT = O(n^d \log n) = O(\log n)$$

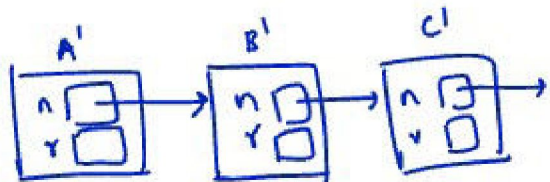


Hash map

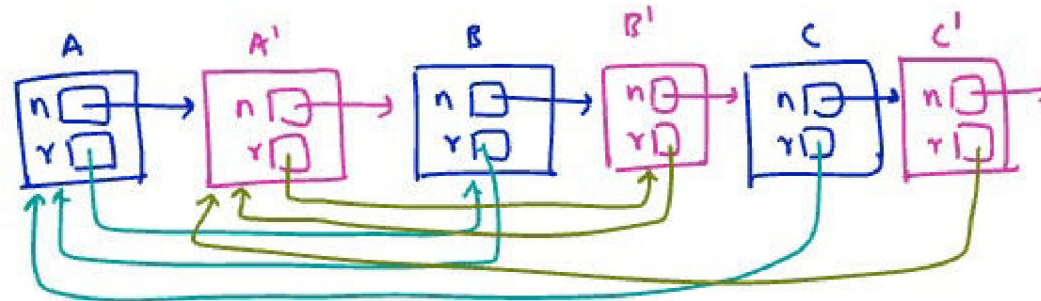
Key	Value
A	A'
B	B'
C	C'



Copy



for each curr in original list
 $\text{map}[\text{curr}].\text{random} = \text{map}[\text{curr}.\text{random}]$



Merge k sorted lists:

- Approach 1: Merge each list one by one into an accumulator list

k - #lists, n - #nodes per list.

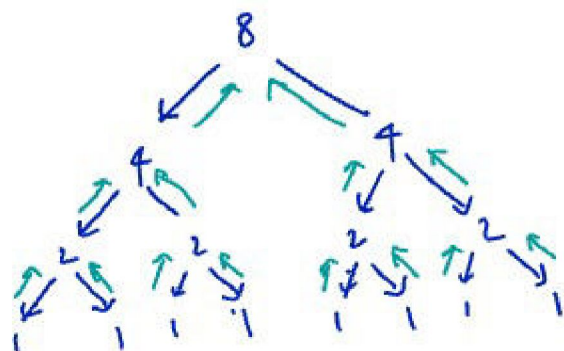
first merge #steps
 $c(2n)$

second merge $c(3n)$

⋮

last merge $c(kn)$

$$\begin{aligned} RT &= cn(2+3+\dots+k) \\ &= cn\left(\frac{k(k+1)}{2} - 1\right) \\ &= \Theta(nk^2) \end{aligned}$$



merge(list 1, list 2)
 ↓ ↓
 m nodes n nodes

$$RT = \Theta(m+n)$$

• $m = \Theta(n) \Rightarrow RT = \Theta(m) \text{ or } \Theta(n)$

• $m \gg n$

(Eg: $n = \log m$) $\Rightarrow RT = \Theta(m)$

$$k = 2^p \Rightarrow p = \log_2 k$$

merge operations

first pass $\frac{k}{2} = 2^{p-1}$

second pass $\frac{k}{4} = 2^{p-2}$

third pass $\frac{k}{8} = 2^{p-3}$

⋮

last pass $1 = 2^0$

work done per merge

$2n$

$4n$

$8n$

⋮

$nk = 2^p n$

Total work done

$2^{p-1} \times 2n = 2^p n$

$2^{p-2} \times 2^2 n = 2^p n$

$2^{p-3} \times 2^3 n = 2^p n$

$2^p n$

$$\Rightarrow \text{Total work} = \# \text{ passes} \times (\text{work per pass}) = (\log_2 k) 2^p n = nk \log k = \Theta(nk \log k)$$

- Approach 4: Create mega list of all lists: nk steps (connect tail of each list to head of next)
- Sort the mega list: $\underbrace{\Theta(nk \log(nk))}_{\text{Total RT}}$

	removeMin()	getMin()	addNew()
Sorted array	$\Theta(n)$	$\Theta(1)$	$\Theta(n)$
Unsorted array	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
Sorted linked list	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$
Heaps	$\Theta(\log n)$	$\Theta(1)$	$\Theta(\log n)$

